Networks and Systems

BSc Computer Science - Durham University

Matthew Emerson

November 2026

Contents

1 Module Introduction **1**
1.1 Assessment . . . 1
1.2 Timetable . . . 1

2 Cyber Security **1**
2.1 Introduction, Threat Modelling, and Applied Cryptography . . . 1
2.2 System Security . . . 4
2.3 Authorisation, Authentication, Accountability and Access Control . . . 6
2.4 Network Security . . . 8
2.5 Software Security . . . 10

3 Networks **11**

4 Databases **12**
4.1 Enhanced Entity-Relationship Model . . . 12
4.2 Semistructured Databases (XML) . . . 12
4.3 XML Data Manipulation (XPath and XQuery) . . . 12
4.4 Transactions . . . 12
4.5 Concurrency Control . . . 12
4.6 Distributed Transactions . . . 12
4.7 Distributed Concurrency Control . . . 12

5 Distributed Systems **12**
5.1 Introduction to Distributed Systems . . . 12
5.2 Middleware and RMI Technologies . . . 12
5.3 Message-Oriented Middleware and Web Services . . . 12
5.4 Replication Models . . . 12
5.5 Concurrent Events and Consistency Control . . . 12
5.6 Fault Tolerance . . . 12
5.7 Load Distribution . . . 12

I Appendix **13**

6 Bibliography **13**
6.1 Module Professors . . . 13
6.2 References . . . 13
6.3 List of Figures . . . 13
6.4 Reading List . . . 13

1 Module Introduction

Networks and Systems has the module code COMP2211 and is led by Dr Wanqing Tu.

1.1 Assessment

This module is assessed through two pieces of summative coursework and two final exams, these can be seen in Table 1. Additional formatives are also shown in italics.

Title	Opening	Due	Feedback	Professor	Weight
Cyber Security	9th Oct	5th Nov	3rd Dec	Dr Carlton Shepherd	25%
Networks	23rd Oct	21st Jan	18th Feb	Dr Wanqing Tu	25%
Dist. Sys. & Databases	May Exam	-	-	Dr Vasantam	50%

Table 1: Assessments for Networks and Systems.

1.2 Timetable

This module is taught through one two-hour lecture and one two-hour computer class each week as seen below in Table 2.

Day	Time	Class Type	Location	Lecturer
Thursday	09:00 - 11:00	Lecture	CLC202	Dr Carlton Shepherd
Thursday	16:00 - 18:00	Computer Class	CC-0007	-

Table 2: Michaelmas Term Timetable for Networks and Systems.

2 Cyber Security

This section is taught by Dr Carlton Shepherd. The recommended reading for this module is as follows:

- Practical Cryptography for Developers (Nakov, Stefanov and Shideroff, 2018), specifically the section on "Encryption: Symmetric and Asymmetric".

2.1 Introduction, Threat Modelling, and Applied Cryptography

2.1.1 Basic Definitions

Definition 2.1.1.1: A system's security depends on the definition of the adversary: who they are, what they can do (or their capabilities) and their determination (time, money, resources). This is the adversary or threat model.

Definition 2.1.1.2: An asset is something that is worth protecting.

Definition 2.1.1.3: A threat is a person or program that may try and access/steal your assets.

Definition 2.1.1.4: An assumption is something you are relying on to be true/secure by default.

Definition 2.1.1.5: Your goal should be in terms of what needs to be secure, and how long for.

Definition 2.1.1.6: The controls are the things you can do to attempt to protect your assets.

The CIA triad are the three main security properties, these are:

- **Definition 2.1.1.7**: Confidentiality: The outcomes of different actions in the system are only visible by authorised subjects.
- **Definition 2.1.1.8**: Integrity: Actions modifying the system can only be performed by authorised subjects.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

- **Definition 2.1.1.9**: Availability: Subjects can perform all actions they are authorised for.

In addition to these properties, it is also important to consider additional properties, such as:

- **Definition 2.1.1.10**: Authenticity: An entity is who they claim they claim to be.
- **Definition 2.1.1.11**: Non-Repudiation: An actor cannot easily deny that an action has occurred.
- **Definition 2.1.1.12**: Context-Binding: An action is valid only for a particular period of time, distance, location, and/or environment.

Definition 2.1.1.13: An exploit is a piece of code, a tool, or a technique that takes advantage of a flaw in software, hardware, or a network to cause unintended behaviour.

Definition 2.1.1.14: A vulnerability is a weakness or flaw in a system's design, implementation, operation, or human behaviour that an attacker can exploit to compromise security.

Definition 2.1.1.15: Risk is the probability or liklihood that a threat will exploit a vulnerability and cause harm, loss, or negative consequences to an asset.

2.1.2 STRIDE Modelling

Definition 2.1.2.1: One method of classifying security risks is STRIDE, which was first developed by Microsoft. It is separated as follows:

- **Definition 2.1.2.1**: **S**poofing is the act of using someone else's credentials to perform an operation and is a failure of identification and authentication.
- **Definition 2.1.2.2**: **T**ampering is the malicious modification of data and is a failure of integrity and authenticity.
- **Definition 2.1.2.3**: **R**epudiation is the act of denying having performed an operation and is a failure of accountability and identification.
- **Definition 2.1.2.4**: **I**nformation Dislosure is the exposure of information to unauthorised users and is a failure of confidentiality.
- **Definition 2.1.2.5**: **D**enial of Service is the act of denying service to legitimate users and is a failure of availability.
- **Definition 2.1.2.6**: **E**levation of Privilege is an instance of an unprivileged user gaining privileged access and is a failure of confinement and authentication.

This is a helpful model for structuring the analysis of potential system threats but it is not exhaustive or exclusive.

2.1.3 Applied Cryptography

Definition 2.1.3.1: To encode some given data, it is transformed from one format/domain to another. This is fully reversible, and does not provide any secrecy. The inverse procedure is called decoding.

- **Definition 2.1.3.2**: A subtype of encoding is steganography, where the existence of the secret message is concealed.

Definition 2.1.3.3: To hash some given data, it is transformed into a (usually) smaller, fixed-size. This is most commonly considered a one-way function. An example of this are SHA algorithms.

- A hashing algorithm works by taking a message m and applying a hashing algorithm $h = H(m)$. h can then be sent securely and the recipient can then take $h' = H(m')$ where $m = m'$, and if $h' = h$, then the message recieved is correct.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

Definition 2.1.3.4: Non-secret data (plaintext) can be encrypted into ciphertext using a secret key. The output is $Enc(m, K) = c$. The inverse procedure, decryption, again uses a secret key to transform the ciphertext back to plaintext $Dec(c, K) = m$.

2.1.4 Cipher Examples

Definition 2.1.4.1: A substitution cipher works by taking each letter in the alphabet and replacing it with another one. This was purportedly created and used by Julius Caesar. These are weak though and can be broken by:

- **Definition 2.1.4.1**: A brute-force attack tries every possible key, which for substitution ciphers becomes trivial.
- **Definition 2.1.4.2**: A frequency analysis attack matches the ciphertext to the known frequency of English letters.

Definition 2.1.4.4: Another historical example of a cipher is a Vigenere cipher. This uses a sliding password, such that repeats of the same letter are coded differently. Attacks against this have to guess possible key lengths and then try all possibilities using frequency analysis. This is much harder to break.

Definition 2.1.4.5: Another encryption method is using one time pads. Here, you select a key as long as your message. Data is converted into binary and bitwise XORed with the key. To decode it, you then bitwise XOR the ciphertext with the key. This is considered theoretically secure as long as the key generation process is truly random, the key is never reused, and the key is kept secret.

Definition 2.1.4.6: Stream ciphers use a short secret key and a nonce - a use-once but public value - to generate a long pseudorandom sequence called a keystream. Only the short secret key must be kept private. We then XOR with the message and keystream.

- To encrypt, you take the secret key K and the nonce N and feed them through a keystream generator. The resulting keystream S is then XORed with the plaintext p to produce ciphertext C.
- To decrypt, the recipient takes the same secret key K and nonce N, feeds them through the keystream generator and then XOR the result with the ciphertext C to produce the original plaintext p.

Definition 2.1.4.7: Block ciphers transform a fixed-size block of plaintext into a ciphertext block of the same size, using a secret key. Encryption applies a keyed transformation to the plaintext block, and decryption reverses the transformation using the same key. An example of this is AES encryption. To encrypt longer messages, we use a mode of operation which specifies how to encrypt message blocks in sequence - this usually involves a nonce (initialisation vector).

- **Definition 2.1.4.7**: AES Electronic Code Book (ECB) mode works by taking the plaintext, splitting it into blocks, and then encrypting each of them separately. This is not secure.
- **Definition 2.1.4.8**: AES Cipher Block Chaining (CBC) mode splits the plaintext into blocks and encrypts the first one. The result is then XORed with the second plaintext block before it is encrypted, and so on.

2.1.5 Public Key Cryptography

Definition 2.1.5.1: Rivest, Shamir, Adelman (RSA) is an algorithm where person A computes the product of two prime numbers $n = p \times q$ and makes the value of n public. Person A then generates a public value e and a private value d using p and q. e can then be used to encrypt messages to send to person A, and d used to decrypt them. This is secure based on the computational difficulty of factorisation, but with the rise of quantum computing, is becoming less secure.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

Definition 2.1.5.2: Digital signatures enables others to check that a message was signed using a private key and has not been altered. To sign a message, the sender uses their private key, and then sends the message and signature to the recipient. To verify the signature, the recipient uses the sender's public key and compares the result to the message.

2.2 System Security

2.2.1 Analysing the Security of Systems

Definition 2.2.1.1: A system is a set of components that interact to achieve a purpose. It generally consists of five components:

- **Definition 2.2.1.2**: Subjects are active entities with an identity and capabilities.
- **Definition 2.2.1.3**: Resources are passive entities with an identity and properties.
- **Definition 2.2.1.4**: Actions are interactions between subjects and resources.
- **Definition 2.2.1.5**: The purpose of a system is what it is used for, and so what security must preserve.
- **Definition 2.2.1.6**: A boundary defines what is inside (what you control), and what is outside.

When considering the security of a system, we should consider four questions:

- **What matters?** An asset and a security property (e.g., confidentiality).
- **Who could cause harm?** An adversary with stated capabilities.
- **Why could they succeed?** A weakness and the assumptions it violates.
- **What changes the outcome?** Controls and countermeasures. What is the remaining risk?

Definition 2.2.1.7: A trust boundary is where data or control passes between parties that trust each other differently (e.g., a phone to a ticketing service).

Definition 2.2.1.8: An attack surface is every entry point an adversary can reach.

Definition 2.2.1.9: A trusted computing base (TCB) is what must work correctly for the goal to hold. If any part fails, security fails.

Definition 2.2.1.10: A reference monitor sits between the subjects and objects. It recieves requests for the objects, compares them with the security policy, and if it passes, returns the requested objects. It is important when designing a system that this is always invoked, it is tamper-proof, and that the check is verifiable.

Actor	Motivation	Typical Capability	Example
Opportunists	Curiosity, notoriety, easy money.	Low: Off-the-shelf tools and mass scanning.	Mirai Botnet (2016).
Hacktivists	Publicity for a cause.	Low-to-Medium: DDoS, leaks, defacement.	Operation Payback (2010).
Organised Criminals	Money.	Medium-to-High: Ransomware, affiliates, access brokers.	Rhysida (2023).
Insiders	Grievance money, or simple mistakes.	High: They already have legitimate access.	Morrisons (2014).
Nation States	Espionage, sabotage, strategic advantage.	Very High: Custom tools, patience, zero-days.	SolarWinds (2020).

Table 3: Potential adversaries to a system.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

Example 2.2.1.1: The Hezbollah pager attack was an attack where Hezbollah ordered thousands of new pagers for members across Lebanon and Syria. This consignment was intercepted by agents in a supply chain attack and 10 to 20 grams of military grade explosive was packed into each device. These pagers were then delivered to Hezbollah and handed out to members. The pagers then recieved a code, buzzed, and exploded in pockets, on belts, or in faces of users.

Definition 2.2.1.11: It can also be useful to consider system security from an adversarial perspective. To do this, people commonly use the kill chain model.

- Reconnaissance: Harvesting email addresses, conference information, etc.
- Weaponisation: Coupling an exploit with a backdoor into a deliverable payload.
- Delivery: Delivering a weaponised bundle to the victim via email, USB, etc.
- Exploitation: Exploiting a vulnerability to execute code on the victim's systems.
- Installation: Installing malware on an asset.
- Command and Control (C2): Setting up a control channel for remote manipulation of victims.
- Actions on Objectives: With "hands on keyboard" access, intruders can accomplish their original goals.

Definition 2.2.1.12: Reconnaissance can be split into a few types: passive gathering where attackers have no interaction with the target, active gathering where they do, and open-source intelligence (OSINT) where the attacker analyses data from publically available sources. They do this to identify potential attack vectors.

Example 2.2.1.2: The British Library attack in 2023 can be split into the categories shown in the Kill chain:

- Reconnaissance: Hostile reconnaissance on the 25th-26th October was detected and blocked, but treated as a one-off.
- Delivery and Exploitation: Most likely privileged credentials obtained by phishing or brute force, then used on a remote-access server without MFA. No software exploit was needed.
- Installation and C2: The attackers hijacked the network's own administration tools, and deleted log files to cover their tracks.
- Actions on Objectives: A "historically complex" legacy network let them reach well beyond the entry point and roughly 600 GB was copied. The servers were then encrypted or destroyed.

2.2.2 Defence

Definition 2.2.2.1: Asymmetry in security is the principle that finding a hole is much easier than securing all possible holes. However, an attack is a sequence of stages that are all opportunities to detect or stop it.

Definition 2.2.2.2: An attack tree is constructed such that: the root is the attackers' goal, the children are the different ways to achieve it, and the leaves are concrete actions. An OR node indicates that any of the children achieves the parent, while an AND node indicates that all children are required to achieve the parent. A set of leaves that satisfies the root is a complete attack.

When designing a system, it is important to consider the following principles: minimise the attack surface, split your defence across many layers, assign users as few privileges as necessary, deny requests as a default reponse unless explicitly allowed, attackers will always take the weakest link, and you should always assume there will be a breach, so monitor and plan for recovery.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

2.3 Authorisation, Authentication, Accountability and Access Control

Definition 2.3.0.1: Identification identifies who someone claims to be.

Definition 2.3.0.2: Authentication tests whether a person can prove their identity.

Definition 2.3.0.3: Authorisation defines what a user is allowed to do.

Definition 2.3.0.4: Access control ensures that a user's authorisation is enforced on every request.

Definition 2.3.0.5: Accountability ensures that actions can be traced back to the user.

2.3.1 Authentication

Definition 2.3.1.1: The authentication lifecycle is separated into five steps. These are:

- **Enrolment** binds an identity to a credential. This can go wrong if an attacker registers as you.
- **Authentication** requires the user to present their credentials for the verifier to check. This can go wrong after brute-force, phishing, or stolen password attacks.
- A **session** sets up a token to stand in for you on every later request. This goes wrong if a stolen cookie skips the login entirely.
- **Recovery** regains access when a credential is lost. This can go wrong if there are guessable security questions, or by exploiting the help-desk.
- **Revocation** removes access when people leave, or when credentials leak. This goes wrong if old codes still work.

Definition 2.3.1.2: Multi-factor authentication can make a system more secure. The factors you can use are: something you know; something you have; and something you are.

Definition 2.3.1.3: No authenticator is perfect, we can judge their accuracy by measuring their false acceptance rate and false rejection rate. Generally, lowering one raises the other. For example, touch ID has a FAR of $1:50,000$ and face ID has a FAR of $1:1,000,000$.

Definition 2.3.1.4: Each authentication attempt gives a similarity score, not a boolean value. The threshold (visualised in Figure 1[1]) turns it into a decision. If we raise the threshold, then there will be fewer false accepts, but more false rejects, and vice-versa.

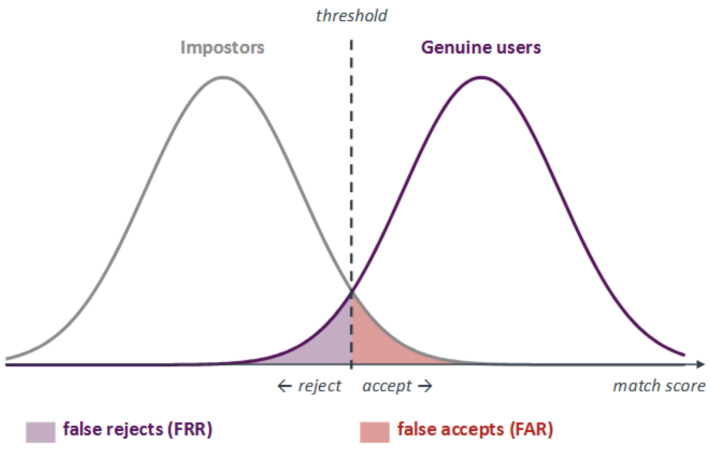


Figure 1: Threshold Trade-off.

[1]Image taken from Deparment of Computer Science (2026).

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

Definition 2.3.1.5: The equal error rate is where $FAR = FRR$. This is good for comparing systems, but is rarely the right operating point. The right setting depends on what each error costs.

Passwords persist because they are free and work anywhere. However, because people choose them, reuse them, and/or type them anywhere, they can become insecure. Some common attacks against passwords are in Table 4.

Attack	How It Works	Main Defence
Online Guessing	Try common passwords against the login page.	Rate limiting, blocklists of common passwords.
Credential Stuffing	Replay uername-password pairs leaked from other sites.	MFA, checks against breached passwords.
Phishing	A look-alike site collects whatever the user types.	Phishing-resistant authentication (passkeys).
Infostealer Malware	Malware copies saved passwords and session cookies.	Endpoint protection, device-bound credentials.
Offline Cracking	Steal the password database, guess at leisure, no rate limit.	Salted, slow password hashing.

Table 4: Common attacks against passwords.

When hashing passwords, you should aim to use a slow hash function so that you can reduce the number of guesses per second, and therefore increase the time required to try all possible passwords. For example, SHA-256 allows for 28 billion guesses per second, and so passwords can be guessed in a maximum of 2 minutes, whereas bcrypt only alows 9.5 thousand guesses per second, therefore increasing the time required to around 9 years.

Definition 2.3.1.6: Entropy measures uncertainty in bits. A secret chosen uniformly from S possibilities has $H = \log_2 S$ bits.

$$H(N, L) = \log_2(N^L) = L \log_2 N \tag{1}$$

where N is the size of the alphabet, L is the length.

Definition 2.3.1.7: In relay attacks, the attacker never breaks the cryptography. Instead, they forward the challenge and the response so both ends belive they are closed. To defend against this, you can introduce distance bounding where you time the challenge-response, or to require a user action.

Multi-factor authentication can be defeated if attackers use one of the techniques in Table 5.

Technique	How It Works
Push Fatigue	Spam approval requests until the user taps “approve”.
Real-Time Phishing Proxy	A fake site relays password and code to the real one, then keeps the session cookie.
SIM Swap	Persuade the mobile operator to move your number to the attacker’s sim.
Help-Desk Social Engineering	Phone IT support as the victim and have MFA reset.
Session Theft	Steal the cookie after login, MFA has already happened.

Table 5: Common attacks against MFA.

2.3.2 Authorisation and Access Control

Definition 2.3.2.1: Access control can be implemented using one of the following approaches:

- Discetionary (DAC) where the owner decides who gets access. Examples of this are Unix permissions, Windows ACLs, or sharing a Google Doc. It is flexible, but anything running as you, including malware, can give your data away.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

- Mandatory (MAC) where a system-wide policy decides and users cannot override it, even for files they create. Examples include government classifications or SELinux. It is rigid byt contains Trojan horses and careless users.
- Caoabilities where unforgeable tokens are held by subjects. It is good for delegation or distributed systems, but can lead to issues with revocation.
- Role-Based (RBAC) where permissions attach to roles which are assigned to users. Joins, moves, and leaves change one assignment, rather than hundreds of permissions.
- Attribute-Based (ABAC) where you decide the access from attributes of the subject, the object, the action, and the environment. This is good because it is fine-grained and context aware, but can lead to compications when trying to work out who can access a given file.

An important principle is that users with a set access level cannot read up (you can only read an object if your clearance is $\geq$ its classification), and cannot write down (write an object only if its classification $\geq$ your clearance.)

Access control can fail when: the user is logged in, but nobody checks that this record is theirs; admin pages are protected only by hiding the link from the menu; or a role/price is in a hidden field, cookie, or JavaScript.

2.3.3 Accountability

When maintaining a system you should have methods for automatically logging: logins (successes and failures); MFA changes, password resets and recovery; role/permission changes; access to sensitive objects and denied requests; and administrative actions.

These logs should be protected by placing them off the host and making them append-only. They should also either use synchronised clocks, or there is no timeline, and should have an alert on patterns. They should also not allow any shared accounts.

2.4 Network Security

When using a network, each hop gives an opportunity for an attacker to read (confidentiality issue), change (integrity issue), drop or flood (availability issue), or fake (authenticity) your traffic.

These attackers can take one of three forms:

- **Definition 2.4.0.1**: An off-path attacker is not on your path. They can send packets with any source address, but cannot see the replies.
- **Definition 2.4.0.2**: A on-path attacker sees every packet. They can read, modify, drop, inject, and/or replay.
- **Definition 2.4.0.3**: A volumetric attacker needs no cleverness, it just sends more traffic than a target system can handle.

2.4.1 Protocols

Each layer of the network stack wraps the one above it in its own header (e.g., data $\rightarrow$ TCP segment $\rightarrow$ IP packet $\rightarrow$ WiFi or Ethernet frame). A header is just bytes that the sender wrote. The original protocols check for accidents (checksums), but these are not necessarily for security. The security was instead added later at different layers:

- The application layer: DNSSEC (signed DNS answers) and HTTPS.
- The transport layer: TLS.
- The network layer: IPsec, WireGuard (VPNs).
- The link layer: WPA2/WPA3 (WiFi).

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

UDP uses a "fire and forget" approach to communication. It has no connection, no ordering, and no retransmission. Examples of its implementations include in DNS, NTP, video calls, and games.

Conversely, TCP is a reliable byte stream. It uses handshakes, sequence numbers, acknowledgements, and retransmission. It is used by the web, email, and SSH.

Definition 2.4.1.1: The handshake system that TCP uses is called the three-way handshake. In practice, it uses the following steps:

1. The client sends a *synchronise* SYN request with a sequence number x.
2. The server responds with a *synchronise acknowledgement* $SYN-ACK$ with a sequence number y and an acknowledgement number $x+1$. An acknowledgement ACK says "I have everything up to here", so each side proves it saw the other's number. Each side counts its bytes from its own random start.
3. After the SYN, the system must remember the client. It does this by keeping one slot in its queue for half-open connections.
4. Once the client recieves the $SYN-ACK$ they send one more ACK with acknowledgement number $y+1$ before sending the data. This means that if y is guessable, an attacker that never sees the $SYN-ACK$ can still send the right ACK.

Definition 2.4.1.2: DNS records are stored in one of a few set types which provide various information about a target host. The common types are:

- Address (A) records which give the IP address for the host.
- AAAA records which provide an IPv6 address for the host.
- Cannonical name (CNAME) records which map one name to another and is used for redirection.
- Mail exchange (MX) records which the server uses for sending/recieving emails.
- Nameserver (NS) records which indicate the authoritative DNS server for this domain.

2.4.2 Flooding Attacks

Definition 2.4.2.1: Denial-of-service attacks max out the bandwidth of a service using UDP floods, reflection, and amplication. The attack also manipulates the states by maliciously filling up look-up tables using DNS cache poisoning and SYN floods. It achieves the denial of service by making each request expensive for the application using HTTP floods on search/login pages. The attack becomes distributed (DDoS) when thousands/millions of sources, either hijacked devices or innocent servers, are used to make the requests.

Definition 2.4.2.2: A DNS cache poisoning attack is used by an off-path attacker who waits for a resolver to make a request to a server, before then flooding the resolver with forged answers before the real one arrives.

Definition 2.4.2.3: The DNSSEC system uses the .root's $DNSKEY$ (the root key, KSK-2024) which is used to vouch for .com which has a DS from .root. Then, example.com uses the DS from .com which authenticates it. This system provides origin authentication and upkeeps the integrity of answers. However it does not provide confidentiality, availability, or protection for the last hop.

Definition 2.4.2.4: A SYN flood attack floods a server with SYN requests with random, spoofed sources. These are cheap to send as they consist of one packet and don't require a reply, but they are costly to recieve as each SYN holds a queue slot until it times out. Once the queue fills up, genuine users will be turned away.

This can be prevented by remembering nothing after a request. Instead, the system designs its sequence number y to be 32-bits, split into t which is 5 bits, MSS which is 3 bits, and then MAC which is constructed from a secret, the client IP, the client port, the server IP, the server port, and t. This means that when a SYN arrives, the server replies with y and stores nothing. A real client would then return $ACK = y+1$. The server can then reconstruct the MAC, and if they match it will establish the connection.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

2.4.3 Cryptography on the Wire

On a network, cryptography is applied in many layers: WiFi (WPA2/WPA3) acts between you and the router; a VPN tunnel acts between you and a VPN server; TLS (HTTPS) works all the way between you and the server; and DNSSEC works between you and the ISP.

Definition 2.4.3.1: TLS acts as the security layer between HTTP and TCP/IP. It ensures confidentiality by encrypting traffic, provides integrity by detecting any changes in transit (AEAD), and authenticates the server by adding a certificate to prove that you reached the name you asked for. It also uses a handshake algorithm, called the TLS 1.3 handshake:

1. The browser sends a *ClientHello* with a key share and cipher suites.
2. The server then responds with *ServerHello* with a key share.
3. These two shares are used to create a shared key where the key parameters are unique to the session and deleted post-use. Stealing the server's key later does not decrypt today's traffic.
4. The server then uses this shared key to send *Certificate*, *CertificateVerify*, and *Finished*.
5. The browser then also sends *Finished*, followed by the application data.

TLS fails in practice when it is implemented wrongly, if the certificate authorities cannot be trusted, if the protocols you are using are downgraded/stripped en-route, or through exploiting the users of the system.

2.4.4 Virtual Private Networks (VPNs), Firewalls, and Segmentation

Definition 2.4.4.1: Virtual private networks (VPNs) are effectively "tunnels through the Internet". They are generally one of:

- IPsec + IKEv2 which is the standard for site-to-site communication and is built into most operating systems.
- TLS-based which is used by OpenVPN and many other vendors' SSL VPNs.
- WireGuard which is a bespoke protocol which has been used in Linux since 2020.

Definition 2.4.4.2: Firewalls are a reference monitor for the network. It checks every incoming packet against the network policy and refuses any that don't meet the standard. It can consist of any from:

- A packet filter which is stateless and checks each packet's addresses, ports, and protocol on its own.
- Connnection tracking which is stateful and remembers connections to allow replies to existing connections.
- The application layer which uses a proxy, NGFW, and WAF to read the content of packets, including HTTP requests, DNS queries, and users.

2.5 Software Security

2.5.1 Software Attackers

Definition 2.5.1.1: A remote software attacker sends input that controls any bytes the program reads: packets, form fields, files, headers, and/or log lines. Examples of this include Heartbleed, SQL injection, and LogShell.

Definition 2.5.1.2: A local software attacker is on the machine and already runs as an ordinary user or app, and wants more privilege. Examples of this include PwnKit, kernel bugs, and phone jailbreaks.

Definition 2.5.1.3: An upstream software attacker is in the supply chain and gets code into your program before you run it, e.g., a library, build server, or update. Examples of this include XZ Utils, SolarWinds, and npm worms.

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

2.5.2 Memory Safety

Definition 2.5.2.1: A buffer overflow attack is where an attacker sends a too large input, running straight over the return address. This means that the attacker can choose the bytes and point the new return address wherever they like. When the input function finishes, the attacker, rather than the programmer, decides what runs next.

Some other common main memory bugs are:

- *Out-of-bounds read*, where the input reads past the end of a buffer, such as with Heartbleed or Crowdstrike.
- *Integer overflow*, where a number wraps around, often a size, such as with Stagefright.
- *Format string*, where an input becomes printf's instructions, such as the iPhone Wi-Fi name.
- *Use after free*, where memory is used after it was given back, such as with many browser zero-days.

Developers can mitigate the risks of these attacks by implementing:

- *Stack canary*, where a random value below the return address is checked before returning. If it has changed, then the program stops.
- *No-execute (NX)*, where data areas are marked as not runnable, so smuggled bytess will not run.
- *ASLR*, where everything loads at random places each run.
- *Control-flow checks*, such as in newer CPUs, which keep a protected copy of each return address.

2.5.3 Injection

To prevent SQL injections, code and data should be kept apart. The query's structure should be fixed first and then the values should travel separately and are only ever data. Similarly, least privilege and a web application firewall should be implemented to restrict what the web app is allowed to run.

2.5.4 Finding and Fixing

A vulnerability falls into one of 5 lifecycle stages: the bug is first written, the bug is found, the vendor is told about the bug, a patch is released, the patch is installed.

2.5.5 Digital Forensics

When examining an incident, you should never change data that may be relied on in court. If you must touch originals, be competent, and able to explain what your actions did. Keep an audit trail – an independent examiner should get the same result. The person in charge is responsible for following the law and these principles.

Similarly, you should consider the order of volatility when collecting data about an incident. CPU registers, cache, and memory may be replaced early, whereas temporary files, disk, remote logs/monitoring, or backups/archives will not.

Evidence of an incident can be: logs about who connected, when, and what they sent; timestamps for when data was created, modified, accessed; deleted files that have not yet been overwritten; and metadata such as authors and edit history.

3 Networks

This section is taught by Dr Wanqing Tu. Check for recommended reading.

4 Databases

This section is taught by Dr Vasantam. The recommended reading for this topic is:

- Connolly, T.M. and Begg, C.E. (2015). Database Systems: A practical approach to design, implementation, and management. 6th edn. Harlow, Essex, England: Pearson Education Limited.

4.1 Enhanced Entity-Relationship Model

4.2 Semistructured Databases (XML)

4.3 XML Data Manipulation (XPath and XQuery)

4.4 Transactions

4.5 Concurrency Control

4.6 Distributed Transactions

4.7 Distributed Concurrency Control

5 Distributed Systems

This section is taught by Dr Vasantam. The recommended reading for this topic is:

- Van Steen, M. and Tanenbaum, A.S. (2025). Distributed Systems. 4th edn. [online] Self-Published. Available at: https://www.distributed-systems.net/index.php/books/ds4/ [Accessed 5 Oct. 2026].

5.1 Introduction to Distributed Systems

5.2 Middleware and RMI Technologies

5.3 Message-Oriented Middleware and Web Services

5.4 Replication Models

5.5 Concurrent Events and Consistency Control

5.6 Fault Tolerance

5.7 Load Distribution

LaTeX File Template v.2.3.0: Copyright 2026, Matthew Emerson

Part I
Appendix

6 Bibliography

6.1 Module Professors

Module Lead: Doctor Wanqing Tu

- Focus: Computer networks and wireless communications.
- Email: wanqing.tu@durham.ac.uk
- Office Hours: Thursday 11:00 - 12:00.

Lecturer: Doctor Thirupathaiah Vasantam

- Focus: Applied probability and performance modelling.
- Email: thirupathaiah.vasantam@durham.ac.uk
- Office Hours: Monday 14:00 - 15:00.

Lecturer: Doctor Carlton Shepherd

- Focus: Cyber security.
- Email: carlton.g.shepherd@durham.ac.uk
- Office Hours: Email to make an appointment.

6.2 References

6.3 List of Figures

6.4 Reading List

6.4.1 Cyber Security

Nakov, S., Stefanov, M. and Shideroff, M. (2018). Practical Cryptography for Developers. [online] sec. Encryption: Symmetric and Asymmetric Available at: https://cryptobook.nakov.com/ [Accessed 9 Oct. 2026].

Department of Computer Science, Durham University (2026). The Threshold Trade-Off. [online] Available on Durham University's Blackboard [Accessed 26 Oct. 2026].

6.4.2 Databases

Connolly, T.M. and Begg, C.E. (2015). Database Systems: A practical approach to design, implementation, and management. 6th edn. Harlow, Essex, England: Pearson Education Limited.

6.4.3 Distributed Systems

Van Steen, M. and Tanenbaum, A.S. (2025). Distributed Systems. 4th edn. [online] Self-Published. Available at: https://www.distributed-systems.net/index.php/books/ds4/ [Accessed 5 Oct. 2026].